

```
library(tidyverse)
```

```
## Warning: package 'readr' was built under R version 4.5.2
```

```
## Warning: package 'lubridate' was built under R version 4.5.2
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.2.0
```

```
## v forcats   1.0.1      v stringr   1.5.2
```

```
## v ggplot2    3.5.2      v tibble    3.3.0
```

```
## v lubridate  1.9.5      v tidyr     1.3.1
```

```
## v purrr      1.2.0
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
elevation_df <- tibble( #lookup table
```

```
  Pop = c("TM2","BH","SC","WL1","WL2","WL3","DPR","IH","YOSE1","LV2","SQ3"),
```

```
  Elev_m = c(379, 511, 422, 1614, 2020, 2138, 1019, 454, 2141, 2500, 2373)
```

```
)
```

```
control_data <- read.csv(
```

```
  "../data/STT0 Viability Data - Y1_Control_lab_stored_seed_viability_data.csv"
```

```
)
```

```
control_data <- control_data %>%
```

```
  left_join(elevation_df, by = "Pop") #join elevation with population
```

```
pop_order <- elevation_df %>%
```

```
  arrange(Elev_m) %>% #sort elevation by ascending order
```

```
  pull(Pop) #extract population in elevation order
```

```
control_data <- control_data %>%
```

```
  mutate(
```

```
    Pop = factor(Pop, levels = pop_order), #order populations by ascending elevation
```

```
    Viability_category = as.numeric(Viability_category)
```

```
)
```

```
control_1 <- control_data %>%
```

```
  mutate(
```

```
    viable = if_else(
```

```
      Viability_category == 1, #assign 1 (viable), 0 (non-viable) based on viability category
```

```
      1, #viable
```

```
      0 #non-viable
```

```
    )
```

```
)
```

```
control_12 <- control_data %>%
```

```
  mutate(
```

```
    viable = if_else(
```

```
      Viability_category %in% c(1, 2), #assign 1 (viable), 0 (non-viable) based on viability category
```

```
      1, #viable
```

```

    0 #non-viable
  )
)

control_123 <- control_data %>%
  mutate(
    viable = if_else(
      Viability_category %in% c(1, 2, 3), #assign 1 (viable), 0 (non-viable) based on viability category
      1, #viable
      0 #non-viable
    )
  )

```

```

population_summary_control_1 <- control_1 %>% #group seeds by population
  group_by(Pop, Elev_m) %>%
  summarise (
    n_seeds = n(), #number of control seeds
    mean_proportion = mean(viable), #mean viable
    SE = sd(viable)/sqrt(n()), #SE of mean
    .groups = "drop"
  ) %>%
  arrange(Elev_m) #order population by elevation

```

```

population_summary_control_12 <- control_12 %>%
  group_by(Pop, Elev_m) %>%
  summarise (
    n_seeds = n(),
    mean_proportion = mean(viable),
    SE = sd(viable)/sqrt(n()),
    .groups = "drop"
  ) %>%
  arrange(Elev_m)

```

```

population_summary_control_123 <- control_123 %>%
  group_by(Pop, Elev_m) %>%
  summarise (
    n_seeds = n(),
    mean_proportion = mean(viable),
    SE = sd(viable)/sqrt(n()),
    .groups = "drop"
  ) %>%
  arrange(Elev_m)

```

```

print(population_summary_control_1)

```

```

## # A tibble: 11 x 5
##   Pop   Elev_m n_seeds mean_proportion     SE
##   <fct> <dbl>   <int>         <dbl> <dbl>
## 1 TM2     379     30         0.467 0.0926
## 2 SC      422     29         0.586 0.0931
## 3 IH      454     30         0.433 0.0920
## 4 BH      511     30         0.433 0.0920
## 5 DPR    1019     28         0.321 0.0899

```

```
## 6 WL1      1614      29      0.0345 0.0345
## 7 WL2      2020      28      0.286 0.0869
## 8 WL3      2138      30      0.1 0.0557
## 9 YOSE1    2141      30      0.333 0.0875
## 10 SQ3     2373       8      0.25 0.164
## 11 LV2     2500      29      0.483 0.0944
```

```
print(population_summary_control_12)
```

```
## # A tibble: 11 x 5
##   Pop   Elev_m n_seeds mean_proportion    SE
##   <fct> <dbl>   <int>         <dbl> <dbl>
## 1 TM2     379     30          0.6 0.0910
## 2 SC      422     29          0.621 0.0917
## 3 IH      454     30          0.7 0.0851
## 4 BH      511     30          0.633 0.0895
## 5 DPR    1019     28          0.571 0.0952
## 6 WL1     1614     29          0.0690 0.0479
## 7 WL2     2020     28          0.357 0.0922
## 8 WL3     2138     30          0.2 0.0743
## 9 YOSE1    2141     30          0.4 0.0910
## 10 SQ3     2373       8          0.5 0.189
## 11 LV2     2500     29          1 0
```

```
print(population_summary_control_123)
```

```
## # A tibble: 11 x 5
##   Pop   Elev_m n_seeds mean_proportion    SE
##   <fct> <dbl>   <int>         <dbl> <dbl>
## 1 TM2     379     30          0.933 0.0463
## 2 SC      422     29          0.828 0.0714
## 3 IH      454     30          0.867 0.0631
## 4 BH      511     30          0.8 0.0743
## 5 DPR    1019     28          0.821 0.0737
## 6 WL1     1614     29          0.552 0.0940
## 7 WL2     2020     28          0.464 0.0960
## 8 WL3     2138     30          0.4 0.0910
## 9 YOSE1    2141     30          0.567 0.0920
## 10 SQ3     2373       8          0.5 0.189
## 11 LV2     2500     29          1 0
```

```
control_summary_1 <- control_1 %>%
  group_by(Pop, Elev_m) %>% #group seeds by population
  summarise (
    trials = n(), #total number of seeds
    events = sum(viable), #total number of viable seeds
    proportion = events / trials, #proportion of viable
    .groups = "drop"
  ) %>%
  arrange (Elev_m)

control_summary_12 <- control_12 %>%
```

```

group_by(Pop, Elev_m) %>% #group seeds by population
summarise (
  trials = n(), #total number of seeds
  events = sum(viable), #total number of viable seeds
  proportion = events / trials, #proportion of viable
  .groups = "drop"
) %>%
arrange (Elev_m)

control_summary_123 <- control_123 %>%
group_by(Pop, Elev_m) %>% #group seeds by population
summarise (
  trials = n(), #total number of seeds
  events = sum(viable), #total number of viable seeds
  proportion = events / trials, #proportion of viable
  .groups = "drop"
) %>%
arrange (Elev_m)

print(control_summary_1)

```

```

## # A tibble: 11 x 5
##   Pop   Elev_m trials events proportion
##   <fct> <dbl>   <int>   <dbl>      <dbl>
## 1 TM2     379     30      14      0.467
## 2 SC      422     29      17      0.586
## 3 IH      454     30      13      0.433
## 4 BH      511     30      13      0.433
## 5 DPR    1019     28       9      0.321
## 6 WL1    1614     29       1     0.0345
## 7 WL2    2020     28       8     0.286
## 8 WL3    2138     30       3      0.1
## 9 YOSE1   2141     30      10     0.333
## 10 SQ3    2373      8       2     0.25
## 11 LV2    2500     29      14     0.483

```

```
print(control_summary_12)
```

```

## # A tibble: 11 x 5
##   Pop   Elev_m trials events proportion
##   <fct> <dbl>   <int>   <dbl>      <dbl>
## 1 TM2     379     30      18      0.6
## 2 SC      422     29      18     0.621
## 3 IH      454     30      21      0.7
## 4 BH      511     30      19     0.633
## 5 DPR    1019     28      16     0.571
## 6 WL1    1614     29       2     0.0690
## 7 WL2    2020     28      10     0.357
## 8 WL3    2138     30       6      0.2
## 9 YOSE1   2141     30      12      0.4
## 10 SQ3    2373      8       4      0.5
## 11 LV2    2500     29      29      1

```

```
print(control_summary_123)
```

```
## # A tibble: 11 x 5
##   Pop   Elev_m trials events proportion
##   <fct> <dbl> <int> <dbl>      <dbl>
## 1 TM2     379     30     28      0.933
## 2 SC      422     29     24      0.828
## 3 IH      454     30     26      0.867
## 4 BH      511     30     24      0.8
## 5 DPR    1019     28     23      0.821
## 6 WL1    1614     29     16      0.552
## 7 WL2    2020     28     13      0.464
## 8 WL3    2138     30     12      0.4
## 9 YOSE1   2141     30     17      0.567
## 10 SQ3    2373      8      4      0.5
## 11 LV2    2500     29     29      1
```

```
lm_control_1 <- lm( #fit linear regression model
  proportion ~ Elev_m, #model viability proportion as a function of elevation
  data = control_summary_1 #use population summary data
)
```

```
lm_control_12 <- lm( #fit linear regression model
  proportion ~ Elev_m, #model viability proportion as a function of elevation
  data = control_summary_12 #use population summary data
)
```

```
lm_control_123 <- lm( #fit linear regression model
  proportion ~ Elev_m, #model viability proportion as a function of elevation
  data = control_summary_123 #use population summary data
)
```

```
print(summary(lm_control_1))
```

```
##
## Call:
## lm(formula = proportion ~ Elev_m, data = control_summary_1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.284645 -0.028914  0.006268  0.045707  0.251642
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.795e-01  8.978e-02   5.340 0.000468 ***
## Elev_m      -9.934e-05  5.478e-05  -1.813 0.103180
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1501 on 9 degrees of freedom
## Multiple R-squared:  0.2676, Adjusted R-squared:  0.1862
## F-statistic: 3.288 on 1 and 9 DF, p-value: 0.1032
```

```
print(summary(lm_control_12))
```

```
##
## Call:
## lm(formula = proportion ~ Elev_m, data = control_summary_12)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.43202 -0.09236  0.03210  0.05459  0.55610
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.050e-01  1.558e-01   3.884  0.00371 **
## Elev_m      -6.444e-05  9.505e-05  -0.678  0.51485
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2604 on 9 degrees of freedom
## Multiple R-squared:  0.04859,    Adjusted R-squared:  -0.05713
## F-statistic: 0.4596 on 1 and 9 DF,  p-value: 0.5149
```

```
print(summary(lm_control_123))
```

```
##
## Call:
## lm(formula = proportion ~ Elev_m, data = control_summary_123)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.20482 -0.09857 -0.02566  0.04900  0.44432
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  8.950e-01  1.092e-01   8.193 1.83e-05 ***
## Elev_m      -1.357e-04  6.665e-05  -2.036  0.0722 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1826 on 9 degrees of freedom
## Multiple R-squared:  0.3154, Adjusted R-squared:  0.2394
## F-statistic: 4.147 on 1 and 9 DF,  p-value: 0.07218
```

```
glm_control_1 <- glm( #fit binomial logistic regression model
  viable ~ Elev_m, #model seed viability as function of elevation
  family = binomial, #use binomial error distribution
  data = control_1 #use individual control seed
)

glm_control_12 <- glm( #fit binomial logistic regression model
  viable ~ Elev_m, #model seed viability as function of elevation
  family = binomial, #use binomial error distribution
  data = control_12 #use individual control seed
```

```

)

glm_control_123 <- glm( #fit binomial logistic regression model
  viable ~ Elev_m, #model seed viability as function of elevation
  family = binomial, #use binomial error distribution
  data = control_123 #use individual control seed
)

print(summary(glm_control_1))

##
## Call:
## glm(formula = viable ~ Elev_m, family = binomial, data = control_1)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.0511732  0.2269034  -0.226  0.82157
## Elev_m      -0.0004533  0.0001526  -2.970  0.00298 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 388.06  on 300  degrees of freedom
## Residual deviance: 379.01  on 299  degrees of freedom
## AIC: 383.01
##
## Number of Fisher Scoring iterations: 4

print(summary(glm_control_12))

```

```

##
## Call:
## glm(formula = viable ~ Elev_m, family = binomial, data = control_12)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.4467592  0.2247092   1.988  0.0468 *
## Elev_m      -0.0002875  0.0001426  -2.015  0.0439 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 417.01  on 300  degrees of freedom
## Residual deviance: 412.91  on 299  degrees of freedom
## AIC: 416.91
##
## Number of Fisher Scoring iterations: 4

print(summary(glm_control_123))

```

```
##
## Call:
## glm(formula = viable ~ Elev_m, family = binomial, data = control_123)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.8804475  0.2870764   6.550 5.74e-11 ***
## Elev_m       -0.0006589  0.0001664  -3.961 7.47e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 358.31  on 300  degrees of freedom
## Residual deviance: 341.66  on 299  degrees of freedom
## AIC: 345.66
##
## Number of Fisher Scoring iterations: 4
```

```
prediction_control_1 <- data.frame(
  Elev_m = seq( #sequence of elevation
    min(control_summary_1$Elev_m), #low elev
    max(control_summary_1$Elev_m), #high elev
    length.out = 100 #100 even spaced value
  )
)
prediction_control_1$predicted <- predict(
  glm_control_1, #fitted glm
  newdata = prediction_control_1, #predict across elevation
  type = "response" #return probabilities
)

prediction_control_12 <- data.frame(
  Elev_m = seq( #sequence of elevation
    min(control_summary_12$Elev_m), #low elev
    max(control_summary_12$Elev_m), #high elev
    length.out = 100 #100 even spaced value
  )
)
prediction_control_12$predicted <- predict(
  glm_control_12,
  newdata = prediction_control_12,
  type = "response"
)

prediction_control_123 <- data.frame(
  Elev_m = seq( #sequence of elevation
    min(control_summary_123$Elev_m), #low elev
    max(control_summary_123$Elev_m), #high elev
    length.out = 100 #100 even spaced value
  )
)
prediction_control_123$predicted <- predict(
  glm_control_123,
```



```

newdata = prediction_control_123,
type = "response"
)

figure_control_1 <- ggplot(
  population_summary_control_1,
  aes(x = Elev_m, y = mean_proportion)
) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(
      ymin= mean_proportion - SE,
      ymax = mean_proportion +SE
    ),
    width = 40
  ) +
  geom_line(
    data = prediction_control_1,
    aes (x = Elev_m, y = predicted),
    linewidth = 1, color ="red"
  ) +
  geom_text( #each point label
    aes(label = Pop),
    vjust = 10,
    size = 3
  ) +
  theme_classic () +
  labs (
    x = "Elevation (m)",
    y = "Proportion Viable"
  )
)

figure_control_12 <- ggplot(
  population_summary_control_12,
  aes(x = Elev_m, y = mean_proportion)
) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(
      ymin= mean_proportion - SE,
      ymax = mean_proportion +SE
    ),
    width = 40
  ) +
  geom_line(
    data = prediction_control_12,
    aes (x = Elev_m, y = predicted),
    linewidth = 1, color ="red"
  ) +
  geom_text( #each point label
    aes(label = Pop),
    vjust = 10,
    size = 3
  )

```

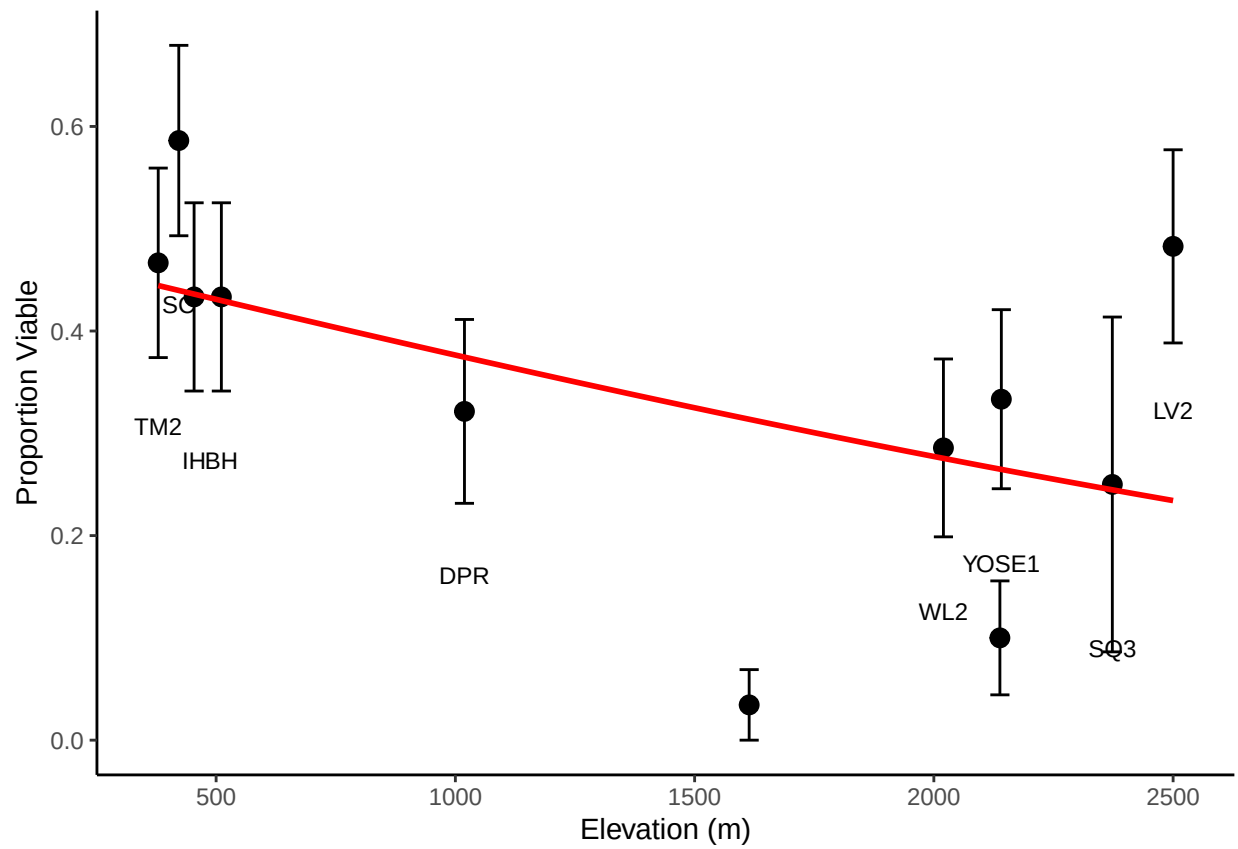
```

) +
theme_classic () +
labs (
  x = "Elevation (m)",
  y = "Proportion Viable"
)

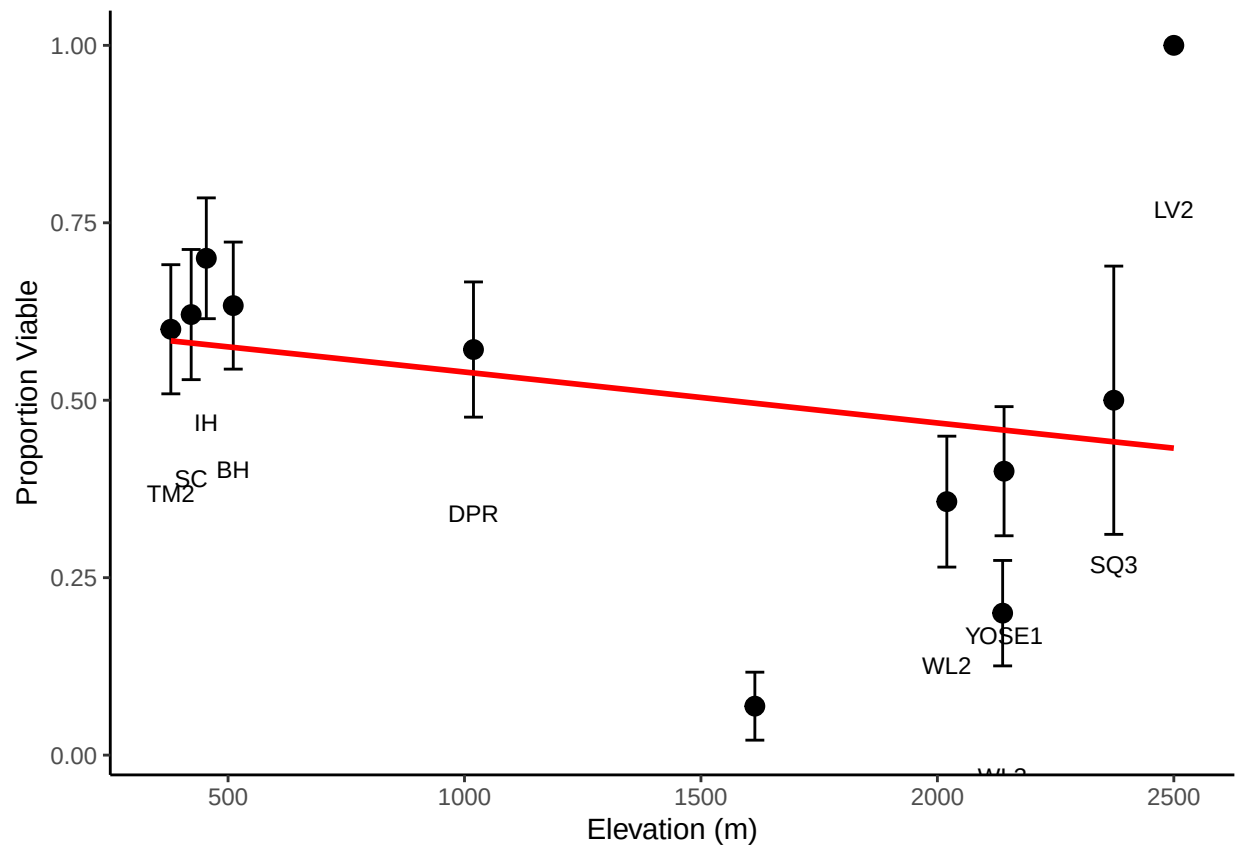
figure_control_123 <- ggplot(
  population_summary_control_123,
  aes(x = Elev_m, y = mean_proportion)
) +
  geom_point(size = 3) +
  geom_errorbar(
    aes(
      ymin= mean_proportion - SE,
      ymax = mean_proportion +SE
    ),
    width = 40
  ) +
  geom_line(
    data = prediction_control_123,
    aes (x = Elev_m, y = predicted),
    linewidth = 1, color ="red"
  ) +
  geom_text( #each point label
    aes(label = Pop),
    vjust = 10,
    size = 3
  ) +
  theme_classic () +
  labs (
    x = "Elevation (m)",
    y = "Proportion Viable"
  )

print(figure_control_1)

```



```
print(figure_control_12)
```



```
print(figure_control_123)
```

